

Isolation Forest

2026-04-17

Introduction

Isolation Forest (Liu, Ting, and Zhou, 2008) is a tree-based unsupervised anomaly-detection method built on a single intuition: outliers are easier to isolate by random splits than densely packed normal points. The algorithm builds many random trees, partitions the data with random feature-and-split-value choices, and uses each observation's average path length to a terminal node as its anomaly score. Short paths mean easy isolation, which means the point is anomalous; long paths mean the point sits in dense regions that take many splits to isolate.

Prerequisites

A working understanding of decision trees, random partitioning, and basic anomaly-detection concepts.

Theory

Build T isolation trees: at each node, pick a random feature and a random split value uniformly in $[\min, \max]$ for that feature. Tree depth is capped at $\log_2 n$ for computational efficiency. For an observation x , the anomaly score is

$$s(x) = 2^{-\mathbb{E}[h(x)]/c(n)},$$

where $h(x)$ is the average path length across trees and $c(n)$ is a normalising constant equal to the average path length of an unsuccessful BST search on n points. Scores close to 1 indicate clear anomalies; scores around 0.5 indicate normal points; scores well below 0.5 are deep in dense regions.

Assumptions

Anomalies are few and distinct from the dense normal points; the subsampling size is sufficient to isolate them ($\psi = 256$ is the canonical default and works well in practice).

R Implementation

```
library(solitude)

set.seed(2026)
n <- 300
df <- data.frame(
  x1 = c(rnorm(n), runif(20, -5, 5)),
  x2 = c(rnorm(n), runif(20, -5, 5))
)

iforest <- isolationForest$new(num_trees = 200)
iforest$fit(df)

scores <- iforest$predict(df)$anomaly_score
```

```
flagged <- scores > quantile(scores, 0.9)

plot(df, col = ifelse(flagged, "red", "grey60"),
     pch = 16, asp = 1,
     main = "Isolation Forest: top-10% scored observations flagged",
     xlab = "x1", ylab = "x2")
```

Output & Results

`isolationForest$predict()` returns a tibble with anomaly scores per observation. Thresholding at a chosen quantile (e.g., the 90th percentile) flags the candidates; the scatter shows flagged points in red and normal points in grey.

Interpretation

A reporting sentence: “Isolation Forest with 200 trees and default subsampling scored 14 of 20 injected anomalies (sensitivity 0.70) at the 90th-percentile threshold, with 16 false positives among the 300 normal points (false-positive rate 5.3 %); the score distribution showed a clear bimodal structure separating anomalies from normal points.” Always state the number of trees, threshold, and per-class accuracy.

Practical Tips

- Tune the contamination threshold (top- k percentile) to match the expected anomaly rate; a 5 % threshold for typical fraud detection, 1 % for very rare events.
- `num_trees = 100-200` is usually sufficient; returns diminish beyond 200 and computational cost grows linearly.
- Isolation Forest handles high-dimensional data well because it does not rely on distance computations; this is one of its main advantages over distance-based methods (kNN-distance, LOF).
- Excellent default choice for anomaly detection; requires no feature scaling and has few hyperparameters to tune.
- For improved separation in non-axis-aligned anomaly structures, Extended Isolation Forest adds random oblique splits; available via the `isotree` package.
- Pair anomaly scores with domain expertise; flagged observations should be investigated, not automatically removed or actioned.

R Packages Used

`solitude` for the canonical R6-based isolation forest interface; `isotree` for Extended Isolation Forest with oblique splits and additional anomaly-detection variants; `dbscan::lof()` for Local Outlier Factor as a distance-based alternative; `mlr3` and `tidymodels` for cross-validated anomaly detection with labelled data.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net

- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis